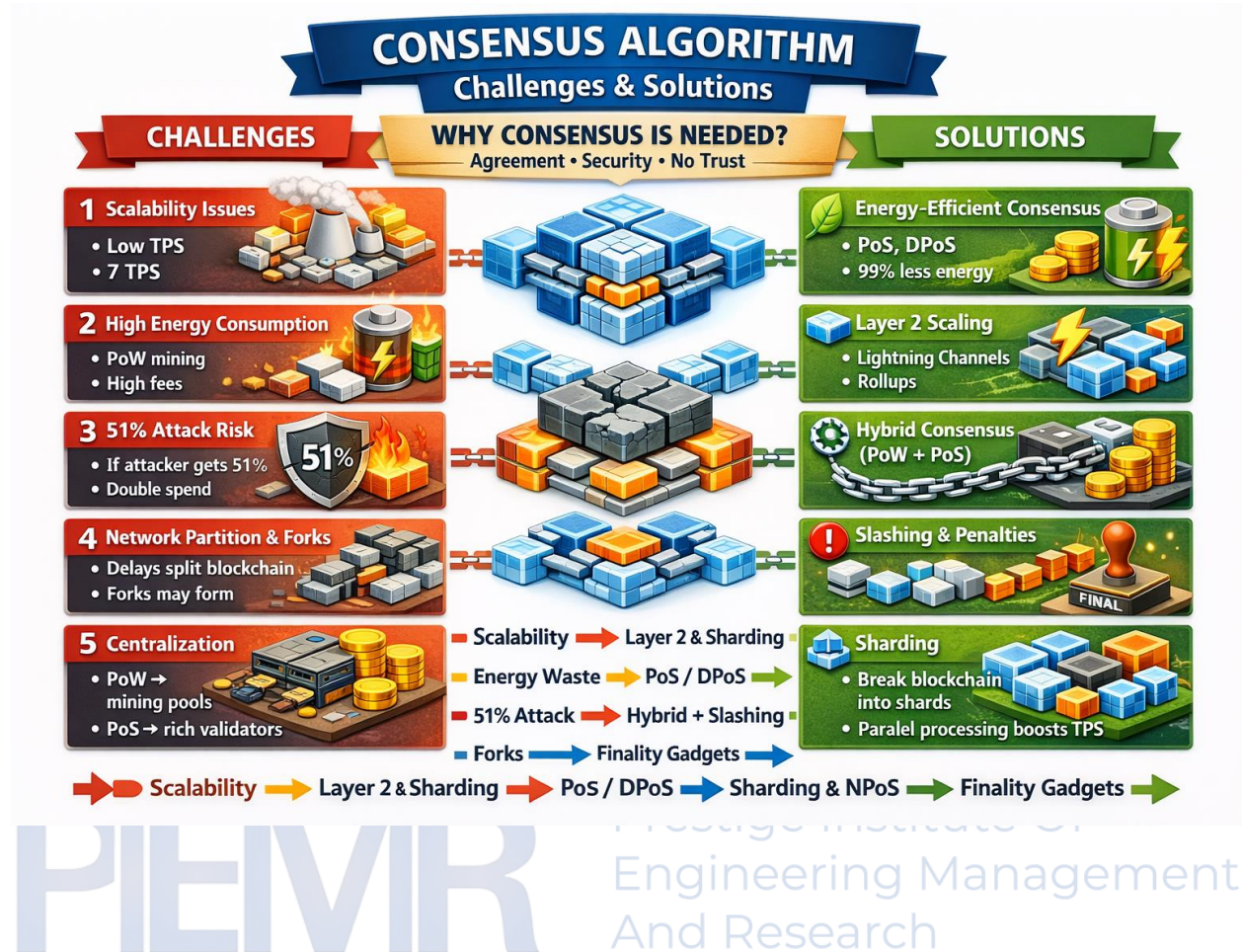


UNIT-II



MODELING FAULTS & ADVERSARIES

• Blockchain in Hostile Environments •

Blockchain in Hostile Environments → Nodes Can Fail, Misbehave, or Attack

A. FAULT MODELS

1 Crash Faults

- Node goes offline due to crash, power failure



2 Omission Faults

- Message not sent/received → network delay or overload



3 Timing Faults

- Node responds too late → deadlines missed → forks possible



4 Byzantine Faults (Most Serious)

- Node behaves maliciously:
 - lies
 - sends fake messages
 - colludes with attackers
 - double spends



B. ADVERSARY MODELS

1 Honest Nodes

- Follow protocol



2 Rational Nodes

- Profit-driven, (may cheat if beneficial)



3 Malicious Nodes

- Intentionally attack blockchain



4 Weak Adversary

- Very limited resources



5 Strong Adversary

- Can delay/reorder messages, control many nodes



Weak Adversary → Adversary → Strong Adversary → Super Adversary

Byzantine Fault Tolerant (BFT) Models

Byzantine Generals Problem

How can all generals (nodes) agree when some may be traitors?

Core Rule: $3f + 1$ Nodes

A blockchain tolerates f malicious nodes only if total nodes $\geq 3f + 1$

1. Classical PBFT

- 3 phases (pre-prepare, prepare, commit)



- 3 phases (pre-prepare, prepare, commit)
- Used in private/consortium blockchains
- Fast finality
- $O(n^2)$ communication → not scalable

2. Federated BFT

- Ripple Stellar
- Quorum slices
- Flexible trust

3. Delegated BFT (dBFT)

- NEO



3. Delegated BFT (dBFT)

- NEO



4. BFT in PoS (Ethereum, Cosmos, Polkadot)

- Validators vote
- 2/3 majority → finality
- Strong security

1. Classical PBFT

- 3 phases (pre-prepare, prepare, commit)
- Used in private/consortium blockchains

2. Federated BFT

- Quorum slices
- Flexible trust
- Fast & scalable



3. BFT in PoS (Ethereum, Cosmos, Polkadot)

- Validators vote
- 2/3 majority → finality
- Strong security



Zero Knowledge Proofs (ZKPs)

Byzantine Generals Problem

Proof that "statement is true" without revealing secret information.

Why Needed in Blockchain?

✓ Privacy

- Hide transaction amount/ address (Zcash)



✓ Regulatory Compliance

- Prove funds exist without showing balance



✓ Scalability

- ZK Rollups → thousands of transactions verified



✓ Secure Voting

- Voter identity hidden, vote verifiable



ZKP Process Summary

1. zk-SNARKs

- Small proofs
- Fast verification
- Needs trusted setup
- Used in Zcash, Ethereum rollups



2. zk-STARKs

- No trusted setup
- Post-quantum secure
- Larger proofs



3. Bulletproofs

- Confidential transactions



3. Bulletproofs

- Confidential transactions
- No trusted setup



→ Commitment → Proof Generation → Verification → ZKP Process Summary →



Prestige Institute Of
Engineering Management
And Research